

Sliding Window

Sunday, May 10, 2026 11:58 AM

It is a method used to solve problems that involve subarrays or substring or window. Instead of iterating repeatedly over the same elements, the sliding window maintains a range that moves step-by-step through the data, updating results incrementally. The main idea is to use the results of the previous window to do computations for the next window.

Example: maximum sum of a subarray with k elements

Time complexity: $O(n)$

Space: $O(1)$

We compute the sum of the first k elements out of n terms using a linear loop and store the sum in a variable.

Then we will traverse linearly over the array till it reaches the end and simultaneously keep track of the maximum sum. To get the current sum of a block of k elements just subtract the first element from the previous block and add the last element of the current block.

```
def maxSum(arr, k):  
    # length of the array  
    n = len(arr)  
  
    # n must be greater than k  
    if n <= k:  
        print("Invalid")  
        return -1  
  
    # Compute sum of first window of size k  
    window_sum = sum(arr[:k])  
  
    # first sum available  
    max_sum = window_sum  
  
    # Compute the sums of remaining windows by  
    # removing first element of previous  
    # window and adding last element of  
    # the current window.  
    for i in range(n - k):  
        window_sum = window_sum - arr[i] + arr[i + k]  
        max_sum = max(window_sum, max_sum)  
  
    return max_sum  
  
if __name__ == "__main__":  
    arr = [5, 2, -1, 0, 3]  
    k = 3  
    print(maxSum(arr, k))
```

There are two steps of sliding window:

- Fixed size sliding window:
 - o Find the size of the window required
 - o Compute the result for the first window

- Then use a loop to slide the window by one and keep computing the results.
- Variable size sliding window:
 - We increase our right pointer one by one till our condition is true
 - At any step if our condition does not match, we shrink the size of our window by increasing left pointer
 - Again, when our condition satisfies, we start increasing the right pointer and follow step 1
 - We follow these steps until we reach the end of the array.

These problems are generally require Maximum/Minimum Subarray, substrings. The size of the subarray or substring will be given in some of the problems. Time complexity can be $O(n)$ or $O(n \log n)$.